

Reporte de Actividades

Mappers

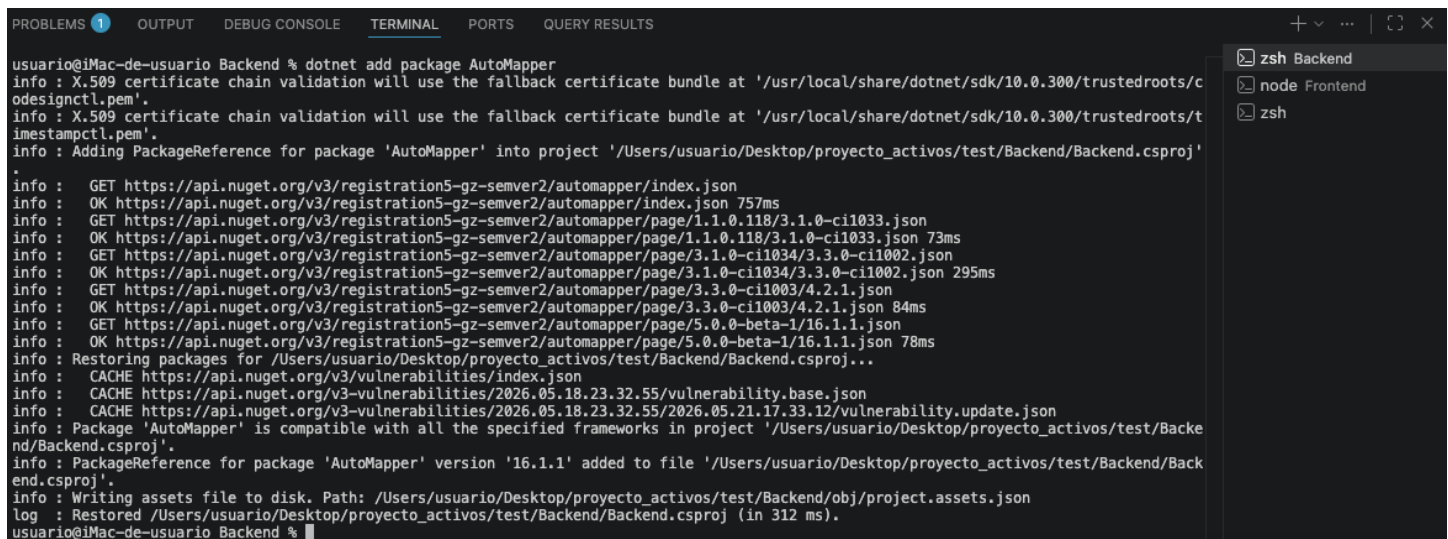
Entendí que un mapper es cuando a un objeto le asignas datos de otro objeto, en este caso seria, el objeto Product que se usa para representar la tabla de la base de datos, y productdto que se usa para mandar los datos al frontend

Hay 3 tipos, el manual directo, donde directamente se hace el mapeo en cualquier objeto; linq projection, que se usa en conjunto con el orm que estoy usando para traer solo las columnas necesarias, esto es lo que venía haciendo desde antes; y extensión method que es hacer un metodo dentro de una clase aparte y llamarlo para ahí mandar lo que es el mapper

Vi que existían algunos paquetes para automatizar esto de los mappers ya que se llega a hacer un codigo grande por asignar manualmente a cada valor a cada parámetro, el más famoso o el más recomendable es el de AutoMapper porque tiene mucha documentación, otro paquete es el de mapster es más rápido y moderno, pero al parecer tiene menos documentación según lo que investigué

Es recomendable usar esto de los mappers automáticos cuando se están usando varios dtos o cuando ya se está repitiendo mucho el mapping, pero si son pocos pues no es tan necesario

Luego para probarlo descargue el automapper ya que con cada consulta a la ia me decía que ese era el que debería usar, pero más adelante como que me empezó a gustar más la idea de mapster, pero primero empecé con este



```
usuario@iMac-de-usuario Backend % dotnet add package AutoMapper
info : X.509 certificate chain validation will use the fallback certificate bundle at '/usr/local/share/dotnet/sdk/10.0.300/trustedroots/codesignctl.pem'.
info : X.509 certificate chain validation will use the fallback certificate bundle at '/usr/local/share/dotnet/sdk/10.0.300/trustedroots/timestampctl.pem'.
info : Adding PackageReference for package 'AutoMapper' into project '/Users/usuario/Desktop/proyecto_activos/test/Backend/Backend.csproj'
info : GET https://api.nuget.org/v3/registration5-gz-semver2/autmapper/index.json
info : OK https://api.nuget.org/v3/registration5-gz-semver2/autmapper/index.json 757ms
info : GET https://api.nuget.org/v3/registration5-gz-semver2/autmapper/page/1.1.0.118/3.1.0-ci1033.json
info : OK https://api.nuget.org/v3/registration5-gz-semver2/autmapper/page/1.1.0.118/3.1.0-ci1033.json 73ms
info : GET https://api.nuget.org/v3/registration5-gz-semver2/autmapper/page/3.1.0-ci1034/3.3.0-ci11002.json
info : OK https://api.nuget.org/v3/registration5-gz-semver2/autmapper/page/3.1.0-ci1034/3.3.0-ci11002.json 295ms
info : GET https://api.nuget.org/v3/registration5-gz-semver2/autmapper/page/3.3.0-ci1003/4.2.1.json
info : OK https://api.nuget.org/v3/registration5-gz-semver2/autmapper/page/3.3.0-ci1003/4.2.1.json 84ms
info : GET https://api.nuget.org/v3/registration5-gz-semver2/autmapper/page/5.0.0-beta-1/16.1.1.json
info : OK https://api.nuget.org/v3/registration5-gz-semver2/autmapper/page/5.0.0-beta-1/16.1.1.json 78ms
info : Restoring packages for /Users/usuario/Desktop/proyecto_activos/test/Backend/Backend.csproj...
info : CACHE https://api.nuget.org/v3/vulnerabilities/index.json
info : CACHE https://api.nuget.org/v3/vulnerabilities/2026.05.18.23.32.55/vulnerability.base.json
info : CACHE https://api.nuget.org/v3/vulnerabilities/2026.05.18.23.32.55/2026.05.21.17.33.12/vulnerability.update.json
info : Package 'AutoMapper' is compatible with all the specified frameworks in project '/Users/usuario/Desktop/proyecto_activos/test/Backend/Backend.csproj'.
info : PackageReference for package 'AutoMapper' version '16.1.1' added to file '/Users/usuario/Desktop/proyecto_activos/test/Backend/Backend.csproj'.
info : Writing assets file to disk. Path: /Users/usuario/Desktop/proyecto_activos/test/Backend/obj/project.assets.json
log  : Restored /Users/usuario/Desktop/proyecto_activos/test/Backend/Backend.csproj (in 312 ms).
usuario@iMac-de-usuario Backend %
```

Aquí automapper usa lo que son los Profiles donde se guarda la config de los mapeos, de que tal objeto se puede convertir en este otro objeto

```
ProductProfile.cs U  Program.cs M  ProductService.cs U
test > Backend > Profiles > ProductProfile.cs > ...
1  using AutoMapper;
2  using Backend.Dtos;
3  using Backend.Models;
4
5  namespace Backend.Profiles;
6
7  2 references
8  public class ProductProfile : Profile
9  {
10     0 references
11     public ProductProfile()
12     {
13         CreateMap<Product, ProductDto>();
14         CreateMap<Product, ProductDetailDto>();
15     }
16 }
```

Y al final se agrega la configuración en program.cs y se cargan los mapeos

```
ProductProfile.cs U  Program.cs M  ProductService.cs U
test > Backend > Program.cs
1  using Backend.Data;
2
3  using Backend.Services;
4  using Microsoft.EntityFrameworkCore;
5
6  var builder = WebApplication.CreateBuilder(args);
7
8  builder.Services.AddDbContext<AdventureWorksContext>(options =>
9  {
10     options.UseSqlServer(
11         builder.Configuration.GetConnectionString("AdventureWorks"),
12         sqlOptions => sqlOptions.CommandTimeout(3));
13
14     builder.Services.AddAutoMapper(config =>
15     {
16         config.AddProfile<ProductProfile>();
17     });
18     builder.Services.AddScoped<ProductService>();
19     builder.Services.AddControllers();
20
21     var app = builder.Build();
22
23     app.MapControllers();
24
25     app.Run();
```

Y pues ya aquí se ve un poco más corta la sintaxis

```
public async Task<List<ProductDto>> GetAllAsync()
{
    var products = await _context.Products
        .AsNoTracking() // este es solo se usa para lectura
        .OrderBy(product => product.Name)
        .Take(10)
        .ToListAsync();
    //
    // return await _context.Products
    //     .AsNoTracking()
    //     .OrderBy(product => product.Name)
    //     .Take(10)
    //     .Select(product => new ProductDto
    //     {
    //         %%      ProductId = product.ProductId,
    //         Name = product.Name,
    //         ProductNumber = product.ProductNumber,
    //         ListPrice = product.ListPrice
    //     })
    //     .ToListAsync();

    return _mapper.Map<List<ProductDto>>(products);
}
```

Lo probe y funciono sin problemas

[←](#) [→](#) [🔄](#) <http://localhost:4200/>

Productos

- Adjustable Race - AR-5381 - \$0 [Ver detalle](#)
- All-Purpose Bike Stand - ST-1401 - \$159 [Ver detalle](#)
- AWC Logo Cap - CA-1098 - \$8.99 [Ver detalle](#)
- BB Ball Bearing - BE-2349 - \$0 [Ver detalle](#)
- Bearing Ball - BA-8327 - \$0 [Ver detalle](#)
- Bike Wash - Dissolver - CL-9009 - \$7.95 [Ver detalle](#)
- Blade - BL-2036 - \$0 [Ver detalle](#)
- Cable Lock - LO-C100 - \$25 [Ver detalle](#)
- Chain - CH-0234 - \$20.24 [Ver detalle](#)
- Chain Stays - CS-2812 - \$0 [Ver detalle](#)

Detalle

ID: 324

Nombre: Chain Stays

Después quise probar mapster y lo instale

```
Application is shutting down...
usuario@iMac-de-usuario Backend % dotnet add package Mapster
info : X.509 certificate chain validation will use the fallback certificate bundle at '/usr/
local/share/dotnet/sdk/10.0.300/trustedroots/codesignctl.pem'.
info : X.509 certificate chain validation will use the fallback certificate bundle at '/usr/
local/share/dotnet/sdk/10.0.300/trustedroots/timestampctl.pem'.
info : Adding PackageReference for package 'Mapster' into project '/Users/usuario/Desktop/pr
oyecto_activos/test/Backend/Backend.csproj'.
info : GET https://api.nuget.org/v3/registration5-gz-semver2/mapster/index.json
info : OK https://api.nuget.org/v3/registration5-gz-semver2/mapster/index.json 109ms
info : Restoring packages for /Users/usuario/Desktop/proyecto_activos/test/Backend/Backend.c
sproj...
info : GET https://api.nuget.org/v3-flatcontainer/mapster/index.json
info : OK https://api.nuget.org/v3-flatcontainer/mapster/index.json 321ms
info : GET https://api.nuget.org/v3-flatcontainer/mapster/10.0.7/mapster.10.0.7.nupkg
info : OK https://api.nuget.org/v3-flatcontainer/mapster/10.0.7/mapster.10.0.7.nupkg 14ms
info : GET https://api.nuget.org/v3-flatcontainer/mapster.core/index.json
info : OK https://api.nuget.org/v3-flatcontainer/mapster.core/index.json 322ms
info : GET https://api.nuget.org/v3-flatcontainer/mapster.core/10.0.7/mapster.core.10.0.7.
nupkg
info : OK https://api.nuget.org/v3-flatcontainer/mapster.core/10.0.7/mapster.core.10.0.7.n
upkg 14ms
info : Installed Mapster.Core 10.0.7 from https://api.nuget.org/v3/index.json to /Users/usua
rio/.nuget/packages/mapster.core/10.0.7 with content hash 73Bd0aZ+t6DoG3X4FYDf0yVgvLxgUXJKIj
rps3W1n7MR03DIWI+sL62VqQSPwuJdn7WJb8HYhkh/BNZ6ZLJydw==.
info : Installed Mapster 10.0.7 from https://api.nuget.org/v3/index.json to /Users/usuario/.
nuget/packages/mapster/10.0.7 with content hash bZgWXYTM21Bc6Xrvk2BzPCFTwnvyTNS04fUli/DVm02K
ya80r5sY9ackpsECVlUqbda7ck9hr/1oKrCyI8yXmw==.
info : GET https://api.nuget.org/v3/vulnerabilities/index.json
```

Y termino viéndose así, ya no fue ni necesario hacer la configuración ni añadirlo al program.cs, que, de ser necesario, si los parámetros llegan a llamarse distinto, puedes hacer la configuración, pero en este caso no necesite hacerlo,

según entendí es como un extensión method entonces no necesitan inyección de dependencias, de ahí investigue que era eso que la verdad no sabía, en resumen es para no tener tanto acoplamiento si una clase necesita de otra clase o servicio, o si en un futuro a la clase se le añade otra clase nueva, como lo tengo en producto service, que tiene adventureworkscontext, luego también vi que habían 3 tipos comunes en .net, add transient para que cada que esa llamada la clase se crea un nuevo objeto cada vez, addscoped que es el mismo objeto en una misma petición http, y el addsingleton es que se usa el mismo objeto, solo se crea un objeto en toda la vida del programa

```
22 public async Task<List<ProductDto>> GetAllAsync()
23 {
24     var products = await _context.Products
25         .AsNoTracking() // este es solo se usa para lectura
26         .OrderBy(product => product.Name)
27         .Take(10)
28         .ToListAsync();
29
30     //lynq projection
31     // return await _context.Products
32     //     .AsNoTracking()
33     //     .OrderBy(product => product.Name)
34     //     .Take(10)
35     //     .Select(product => new ProductDto
36     //     {
37     //         ProductId = product.ProductId,
38     //         Name = product.Name,
39     //         ProductNumber = product.ProductNumber,
40     //         ListPrice = product.ListPrice
41     //     })
42     //     .ToListAsync();
43
44     // automaper
45
46     // return _mapper.Map<List<ProductDto>>(products);
47
48     return products.Adapt<List<ProductDto>>();
49 }
50
```

Y de igual forma funciono

Productos

- Adjustable Race - AR-5381 - \$0 [Ver detalle](#)
- All-Purpose Bike Stand - ST-1401 - \$159 [Ver detalle](#)
- AWC Logo Cap - CA-1098 - \$8.99 [Ver detalle](#)
- BB Ball Bearing - BE-2349 - \$0 [Ver detalle](#)
- Bearing Ball - BA-8327 - \$0 [Ver detalle](#)
- Bike Wash - Dissolver - CL-9009 - \$7.95 [Ver detalle](#)
- Blade - BL-2036 - \$0 [Ver detalle](#)
- Cable Lock - LO-C100 - \$25 [Ver detalle](#)
- Chain - CH-0234 - \$20.24 [Ver detalle](#)
- Chain Stays - CS-2812 - \$0 [Ver detalle](#)

Detalle

ID: 324

Nombre: Chain Stays

SKU: CS-2812